

# Deep Learning for Quadratic Hedging in Incomplete Jump Markets

## Introduction to the Paper

Agram, Øksendal, and Rems (2024) address the problem of hedging options in incomplete jump-diffusion markets, where sudden price discontinuities make perfect replication impossible. Classical models like Black–Scholes assume continuous paths and complete markets, but real assets exhibit sudden jumps. Merton’s (1976) closed-form solution handles jumps by assuming jump risk is diversifiable — an assumption that fails for systematic shocks. More general models such as Kou (2002) offer no closed-form hedging solution at all, making machine learning a natural alternative.

## What the Paper Introduced

The paper proposes a *quadratic hedging* framework: since perfect replication is impossible in incomplete markets, one minimises the expected squared hedging error. For a self-financing portfolio  $\pi(t)$  with initial wealth  $X(0) = z$ , the wealth dynamics are:

$$dX(t) = X(t) \pi(t) \left[ \alpha(t) dt + \sigma(t) dB(t) + \int_{\mathbb{R}^*} \gamma(t, \zeta) \tilde{N}(dt, d\zeta) \right], \quad (1)$$

where  $B(t)$  is a Brownian motion and  $\tilde{N}(dt, d\zeta)$  is a compensated Poisson random measure capturing jumps. The market is incomplete because the jumps introduce unhedgeable risk beyond what a single risky asset can absorb. The optimal pair  $(\hat{z}, \hat{\pi})$  is found via a *Stackelberg game*: the leader sets initial capital  $z$ , the follower picks the best portfolio  $\hat{\pi}_z$ , and then  $\hat{z}$  is chosen to minimise the overall objective. The resulting *minimal variance price* satisfies  $\hat{z} = E_{Q^*}[F]$ , where  $Q^*$  is an explicitly identified positive equivalent martingale measure.

## Deep Learning and LSTM Networks

Because the optimal strategy cannot always be computed analytically, the paper proposes a deep learning algorithm. Time is discretised into  $R$  steps, and at each step a neural network predicts  $\pi(t_i)$  given the current wealth state. The paper compares feed-forward and recurrent architectures, finding that *long short-term memory* (LSTM) networks perform best. LSTMs maintain a memory state across time steps, making them well-suited to financial time series where past jumps contain relevant information. A further advantage is that LSTMs naturally extend to non-Markovian settings, which is important when the optimal decision depends on the full history of the process.

## Minimal Variance Hedging and the Loss Function

The network is trained by minimising the empirical hedging loss over  $M$  simulated paths:

$$\text{Loss} = \frac{1}{2M} \sum_{j=1}^M \left( x_R^{(j)} - F^{(j)} \right)^2, \quad (2)$$

where  $x_R^{(j)}$  is the terminal wealth and  $F^{(j)} = (s_R^{(j)} - K)^+$  is the call option payoff. This directly approximates  $E\left[\frac{1}{2}(X(T) - F)^2\right]$ . In an incomplete market the loss cannot reach zero; residual jump risk always remains. The option price  $\hat{z}$  is the initial capital consistent with the minimising strategy.

## Applications to the Merton and Kou Models

The algorithm is tested on three models: Black–Scholes (complete market benchmark), Merton (log-normal jumps), and Kou (double-exponential jumps). For Merton, the minimal variance price is typically higher than Merton’s original price because it does not assume jump risk is diversifiable, but it produces a symmetric loss distribution and avoids the occasional large losses arising under Merton’s approach. For the Kou model, no closed-form benchmark exists — exactly the setting where deep learning is indispensable, since the network requires only the ability to simulate paths.

## Empirical Results

The algorithm performs well across all models. In the Black–Scholes case, predicted prices and portfolios converge close to the analytical solution (minimum loss  $\approx 10^{-6}$ ). In the Merton model, the algorithm predicts a price of 0.521 versus the Monte Carlo benchmark of 0.519 and Merton’s 0.515. For the Kou model, the predicted price of 0.4987 is consistent with theoretical expectations. Scalability experiments confirm good performance across increasing input dimensions, with accuracy decreasing somewhat for longer maturities due to accumulating discretisation errors.

## Strengths of the Paper

The paper combines rigorous stochastic calculus with practical computation. The Stackelberg game derivation provides a clear theoretical basis, and the LSTM architecture is well-motivated by the sequential nature of the problem. The framework handles multi-dimensional and non-Markovian settings, and the direct comparison between the minimal variance and Merton approaches offers useful practical insight into how jump risk assumptions affect both prices and hedging losses.

## **Limitations of the Paper**

The main limitations are computational cost — the Merton model required 7000 training epochs — and the black-box nature of neural networks, which makes the learned strategy hard to interpret compared to an explicit formula. The paper also does not address calibration to real market data, and performance deteriorates for longer maturities due to accumulated discretisation error.

## **Connection to Our Project**

Our project examines QQQ index option pricing and the volatility smile using classical Merton and Kou jump-diffusion models. The paper by Agram et al. (2024) is a direct computational extension of this framework. When analytical formulas become unavailable — for instance, in stochastic volatility extensions — the LSTM approach provides a viable path forward. The comparison between minimal variance and Merton pricing is also relevant to our analysis, since it clarifies how different assumptions about jump risk premia shape the implied volatility surface and the interpretation of calibrated parameters. Our project can therefore be seen as the classical foundation that this deep learning methodology builds upon.